Tox21 Molecular Toxicity Prediction Using an MLOps Deployment Pipeline

Author Name: Talha Rashid
Department / Program: Data Science
Institution: NUCES, Islamabad
Email: i211365 Github-Link: https://github.com/rTalhaa/MLops-Deployment

***Abstract*—This report presents an end-to-end MLOps pipeline for molecular toxicity prediction using the Tox21 dataset. The project converts molecular SMILES strings into 1024-bit Morgan fingerprints with RDKit, trains and compares Logistic Regression, Random Forest, and Extra Trees classifiers, tracks experiments with MLflow, serves the selected model through FastAPI, and exposes operational metrics through Prometheus and Grafana. The selected model predicts the SR-ARE toxicity endpoint and is packaged with Docker, validated through GitHub Actions CI, and published through a GitHub Container Registry image workflow.**

***Index Terms*—MLOps, Tox21, molecular toxicity prediction, FastAPI, Docker, MLflow, Prometheus, Grafana, GitHub Actions**

I. Introduction

Molecular toxicity prediction is an important machine learning use case in computational drug discovery and chemical safety screening. This project focuses on building a reproducible MLOps workflow around a Tox21 binary classification task rather than only training a standalone model.

The implemented system predicts the `SR-ARE` endpoint from molecular SMILES strings. The repository includes the dataset, training pipeline, selected model artifact, FastAPI inference service, Docker configuration, Prometheus metrics, Grafana dashboard provisioning, CI validation, and CD image publishing workflow.

II. System Architecture

Fig. 1 shows the high-level architecture of the implemented pipeline. The workflow is divided into data and model development, deployment and orchestration, and client and observability components.

The training stage reads `data/tox21.csv`, preprocesses molecules, generates Morgan fingerprints, trains three models, logs runs to MLflow, and saves the best model artifact. The serving stage uses FastAPI inside a Docker-based runtime. Prometheus scrapes the API's `/metrics` endpoint, and Grafana visualizes service health, prediction traffic, request quality, and latency.

III. Dataset and Feature Engineering

The project uses the Tox21 dataset stored as a CSV file. The training script selects the `SR-ARE` target column and removes records with missing target labels. SMILES strings are parsed using RDKit. Molecules that cannot be parsed are skipped during training.

Each valid molecule is represented as a 1024-bit Morgan fingerprint with radius 2. The same fingerprint generation logic is used by the FastAPI service at inference time, ensuring that the training and serving paths use the same feature representation.

TABLE I
Dataset and Feature Configuration

Item	Value
Dataset	Tox21
Input format	CSV
Target column	`SR-ARE`
Molecular input	SMILES string
Feature type	Morgan fingerprints
Fingerprint size	1024 bits
Fingerprint radius	2

IV. Model Training and Selection

The training pipeline compares three classical machine learning models: Logistic Regression, Random Forest, and Extra Trees. A stratified train-test split is used, and model performance is evaluated using accuracy, precision, recall, F1 score, and ROC-AUC.

The selected model is Extra Trees, chosen using ROC-AUC as the selection metric. The selected model is saved as `models/tox21_best_model.joblib`, and metadata is stored in `models/model_metadata.json`.

TABLE II
Model Evaluation Results

Model	Acc.	Prec.	Rec.	F1	AUC
Logistic Regression	0.7494	0.3443	0.6117	0.4406	0.7443
Random Forest	0.7923	0.3846	0.4787	0.4265	0.7592
Extra Trees	0.7700	0.3571	0.5319	0.4274	0.7648

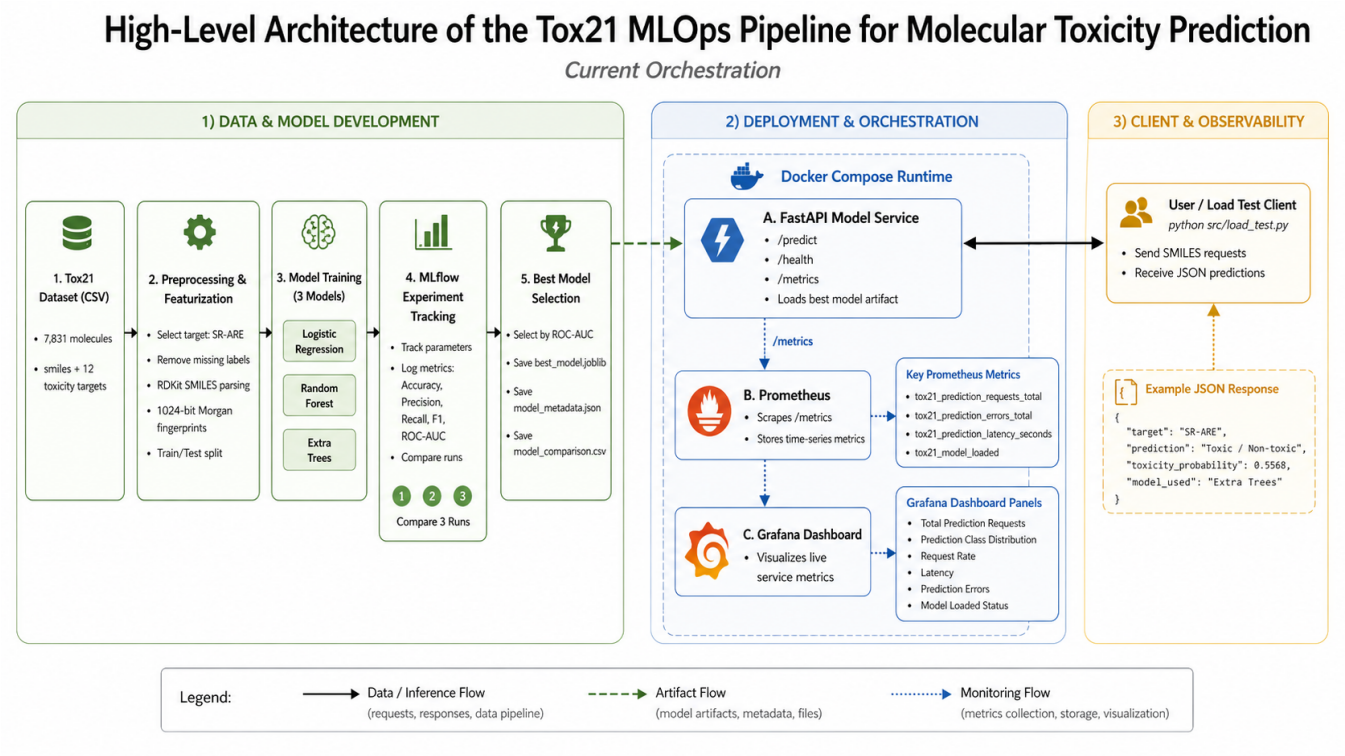

Fig. 1. High-level architecture of the Tox21 MLOps pipeline for molecular toxicity prediction.

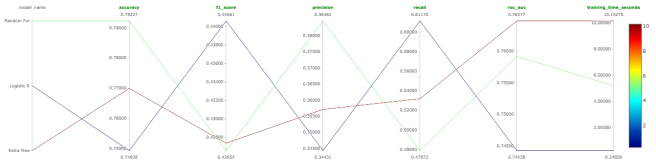

Fig. 2. MLflow experiment tracking screenshot captured from the project artifacts.

V. API Serving Layer

The inference service is implemented with FastAPI in `app/main.py`. The API loads the selected model artifact at startup and exposes the endpoints shown in Table III.

TABLE III
FastAPI Endpoints

Endpoint	Purpose
`GET /`	Service root message
`GET /health`	Model and service health metadata
`GET /version`	App, model, and runtime version metadata
`POST /predict`	Toxicity prediction for a SMILES input
`GET /metrics`	Prometheus-compatible service metrics

The `/predict` endpoint accepts a JSON payload containing a SMILES string. Valid inputs return the target, predicted toxicity label, probability estimate, selected model name, and model version. Invalid SMILES strings are rejected with an HTTP 400 response.

VI. Observability

The API exposes Prometheus metrics for prediction attempts, prediction outcomes, prediction errors, response status codes, prediction latency, and model load status. These metrics support operational visibility into request volume, invalid inputs, server errors, and latency behavior.

Grafana provisioning is included in the repository. The dashboard is grouped around service health, traffic quality, and latency so that panels reflect the available metrics rather than arbitrary visualizations.

Fig. 3 shows the current dashboard view included with the project artifacts.

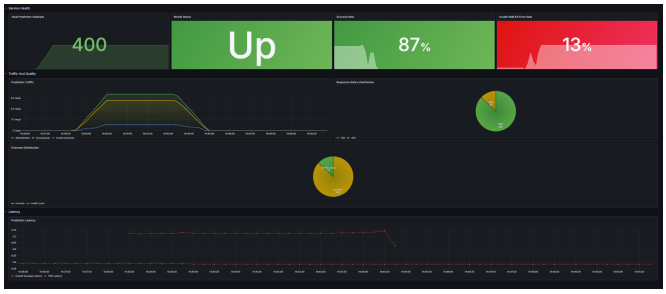

Fig. 3. Grafana dashboard screenshot captured from the project artifacts.

VII. CI/CD and Reproducibility

GitHub Actions is used for CI and CD. The CI workflow runs on pushes and pull requests targeting `main`. It installs

TABLE IV
PROMETHEUS METRICS EXPOSED BY THE API

Metric	Purpose
tox21_prediction_requests_total	Successful predictions by label
tox21_prediction_attempts_total	Total prediction attempts
tox21_prediction_outcomes_total	Outcomes such as success and invalid input
tox21_prediction_errors_total	Prediction errors by type
tox21_prediction_status_codes_total	Responses by HTTP status code
tox21_prediction_latency_seconds	Prediction latency histogram
tox21_model_loaded	Model load status gauge

pinned dependencies, verifies required files, compiles Python files, runs tests, checks model artifact compatibility with the runtime scikit-learn version, and builds the Docker image on pushes.

The CD workflow publishes the API Docker image to GitHub Container Registry. It supports branch, tag, semantic version, latest, and SHA-based image tags. The workflow also includes an optional deploy webhook step controlled by the DEPLOY_WEBHOOK_URL secret.

Runtime reproducibility is improved by pinning dependencies and recording training library versions in models/model_metadata.json. The current model metadata records Python 3.11.9 and scikit-learn 1.8.0 for the training environment, and the API reports both training and runtime scikit-learn versions.

VIII. LIMITATIONS

This project is intended as an MLOps demonstration and should not be treated as a validated toxicology decision system. The model is specific to the SR-ARE endpoint, uses fixed molecular fingerprints rather than learned molecular representations, and reports moderate evaluation metrics. Any real-world scientific, clinical, regulatory, or safety-critical use would require stronger validation, calibration analysis, monitoring, data governance, and domain expert review.

IX. CONCLUSION

The project demonstrates a complete MLOps workflow for molecular toxicity prediction: data preprocessing, feature generation, model comparison, experiment tracking, artifact management, API serving, containerization, observability, CI validation, and image publishing. The implemented pipeline provides a reproducible foundation for demonstrating how machine learning models can be operationalized beyond notebook-based experimentation.

MISSING INFORMATION TO COMPLETE FINAL SUBMISSION

The following academic metadata was not available in the repository and should be filled before final submission:

- Author name
- Institution or department
- Course or module name
- Instructor or supervisor name
- Submission date
- Required citation style or references, if mandated by the course